

INF3200-Distributed Systems Fundamentals

Assignment 3

Distributed Hash Table: Dynamic Membership

Magnus Dahl-Hansen

10.october – 3.november 2025

Contents

1	Introduction	2
2	Implementation	2
2.1	Nodes joining the ring	2
2.2	Nodes leaving the ring	2
2.3	Stabilizing the ring	3
2.4	Simulating nodes crashing	3
2.5	Nodes recovering themselves	3
3	Evaluating the implementation	3
3.1	Setup	3
3.2	Results	4
4	Conclusion	6

1 Introduction

The purpose of this assignment was to implement a distributed hash table (DHT) network based on the Chord protocol, allowing nodes to dynamically join and leave the system. The network was required to handle unexpected node crashes and be able to stabilize itself to maintain a functional ring structure.

2 Implementation

The implementation is separated into five main components: nodes joining the ring, nodes leaving the ring, ring stabilization, simulating node crashes and nodes recovering themselves.

2.1 Nodes joining the ring

Two functions are implemented to enable a new node to join an existing Chord ring. One function manages the logic executed by the new node, while the other handles the logic on the existing node that receives the join request.

When a new node joins the ring, it sends a request to an existing node asking for its successor list and predecessor. The new node then copies this successor list, placing the contacted node at the first position, and then sets the given predecessor as its own predecessor.

On the receiving side, a new ID is received in the join-request. Several checks are performed to determine whether the new ID is the successor or predecessor of the existing node. If this is the case, the existing node returns its successor list and predecessor to the joining node. Otherwise, the request is forwarded to the successor node or to the closest preceding node in the finger table, which is more likely to be responsible for the new ID.

2.2 Nodes leaving the ring

When a node leaves the ring, it sends its successor list to its predecessor and attempts to send its predecessor to one of the nodes in its successor list. After completing these tasks, the node marks its state as "not member" to indicate that it is not part of the ring, but is identifiable as a former member.

A node receiving a leave-call is provided an ID from the departing node. Checks are then made to see if the leaving node is its predecessor or successor. If so, the node updates either its predecessor or successor list based on the case. If the leaving node is neither a predecessor or successor, then the leave-call is ignored.

2.3 Stabilizing the ring

To maintain a consistent ring structure each node runs a stabilization loop every two seconds. In this loop the node sends its own ID to its successor and receives the predecessor and successor list from that successor. If the received predecessor turns out to be the correct successor of the current node, then the node updates its successor list accordingly. Additionally, the node sends a ping to its predecessor to verify if it is still active. If a response is not received, then the predecessor node is assumed to have crashed and is removed.

When a node receives a stabilization message containing the ID of another node, it checks if this node should become its new predecessor. If so, it updates its predecessor. In both cases, the node returns both its predecessor and successor list to the contacting node.

During each stabilization, the node also updates its finger table. For each entry in the finger table an approximate ID is calculated and a lookup request to find the node responsible for this ID is sent to either the closest entry in the finger table or to its immediate successor. If a response is received, then the entry in the finger table is updated with the new responsible node.

2.4 Simulating nodes crashing

To simulate a node being crashed, each function in the implementation includes a state check. If a nodes state is set to *"dead"* or *"not member"*, then the node simulates being either crashed or have voluntarily left the ring. In both cases the node does not process any request and will only respond with a HTTP 500 error message.

2.5 Nodes recovering themselves

When a node recovers from a crash, it is assumed that it still has knowledge of other nodes through entries in its successor list, finger table or its predecessor. These entries are combined into a single list, and the recovering node sequentially tries to send a join-request to each of them. The first node to respond becomes the entry point to rejoin the ring.

3 Evaluating the implementation

3.1 Setup

To evaluate the network implementation, three tests were conducted.

1. Join test: measuring the time it takes for the ring to become stable after 2, 4, 8, 16 and 32 nodes joins the ring.

2. Leave test: measuring how long it takes for the ring to become stable after half the nodes voluntarily leaves the ring, for network sizes of 4, 8, 16 and 32 nodes.
3. Crash test: measuring how long it takes the ring to become stable after a given number of nodes crashes, with test sets ranging from 1 to 30 nodes.

The join and leave tests were performed three times for every network size, and the corresponding standard deviation was calculated. For the crash tests only a single trial was conducted for each number of crashed nodes and there was a 1 second delay between each node crash.

The network is considered stable when every node can reach all other nodes through their successors. To determine this, a stability function was implemented. This function sends a request to a given node, which then forwards the request to its successor until the request returns to the origin node. Each request carries a counter which is incremented at every node it visits, and a list in which each node adds their ID, if the ID is not already in the list. Once the request returns to the origin node the network is considered to be stable if the HTTP response status is 200, the list contains the IDs of all active nodes and the counter matches the total number of active nodes. The test file periodically runs this function every 2 seconds to determine when the ring is stable.

3.2 Results

The results from the join tests (shown in red in figure 1) were quite interesting. As expected, the time for the network to become stable increases with the number of nodes joining. However, it was fascinating to observe that the network stabilizes faster when 16 nodes joined compared to 8 nodes. Several factors could explain this behavior since the stabilization time does not only depend on the number of nodes joining. The timing of the join request matters a lot because of how unstable or busy the network is at that moment, which delays processing of the join request. Furthermore, if the nodes IDs are irregular and the nodes join in a non-sequential way, it will take longer time to locate the correct successors and predecessors compared to an ordered and sequential setup. Larger network sizes also contribute to making these scenarios more likely.

When looking at the results from the leaving tests (shown in blue in figure 1), the observations correspond well with the expectations. The stabilization time increases with the number of nodes leaving the ring and remains significantly lower compared to when nodes join the ring. This is because departing nodes can inform their neighbors about their departure, allowing the network to stabilize faster. Compared to a join request, the network does not need to search extensively for new successors and predecessors, and less information needs to be distributed. Additionally, finger tables does not need to be rebuilt completely, which furthermore reduces the stabilization time.

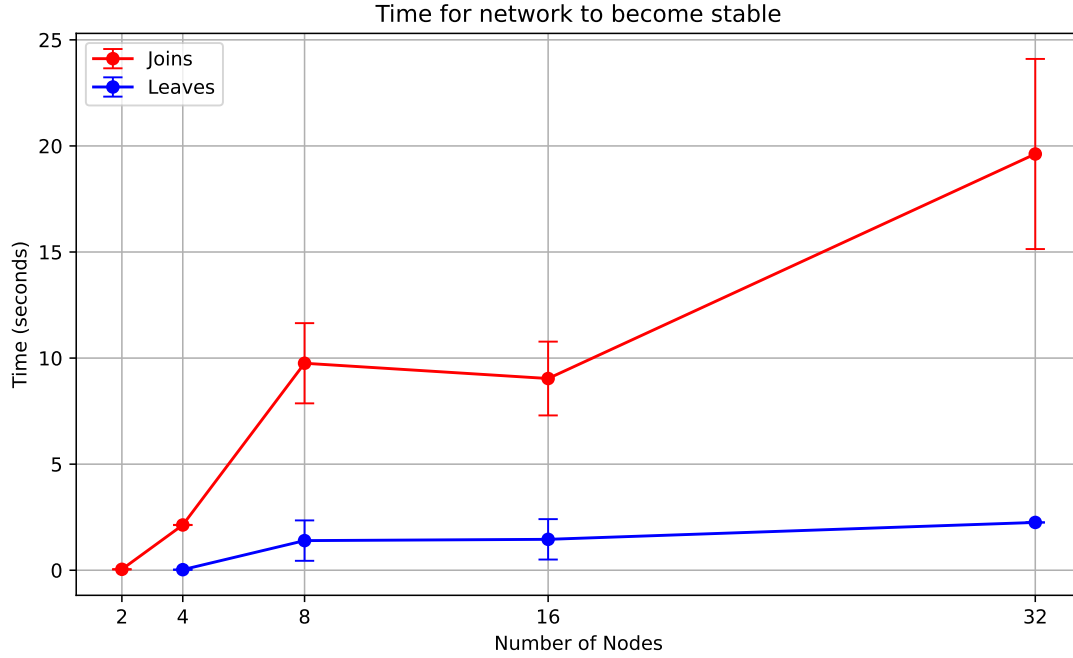


Figure 1

The crash test results (shown in figure 2) revealed an interesting pattern. The stabilization time increased gradually when 1-10 nodes crashed, then became very unstable when 11-21 nodes crashed, before returning to a predictable increase again from 22 crashes and onward. This behavior suggests that the system can stabilize fairly easy with failures from 1-10 nodes and 22+ nodes, but hits a critical point at 11-21 nodes crashing. When 11-21 nodes crash, the system appears to hit a failure point where nodes fail faster than the successor lists and finger tables can recover, causing the ring to fragment and require many more stabilization rounds to become stable again, making the stabilization behave unpredictable.

This unstable behavior might be due to the length of the successor lists being too short causing all immediate successors to be lost or make the finger table entries invalid, forcing nodes to spend more time finding new successors. Another factor is timing. Since crashes occur every second while stabilization rounds happen every two seconds, the ring might be in the middle of stabilizing previous crashes when new crashes occur, furthermore making the stabilization times become unpredictable.

When 22 or more nodes crash, the stabilization times become more predictable again. At this point in time, so many nodes are not responding anymore and the ring has formed a smaller coherent ring. With fewer nodes to manage the network has an easier time finding successors and predecessors, therefore stabilization times becomes faster

and more predictable.

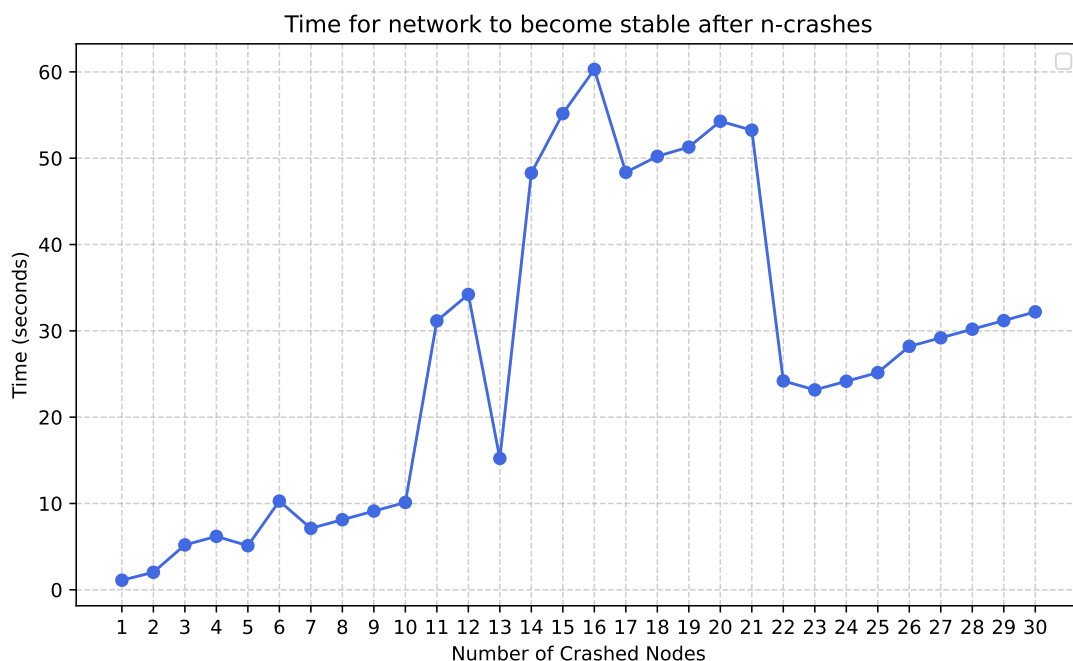


Figure 2: Figure 2

Finally, both the provided API test and the join experiment test pass successfully. however, a minor adjustment was made to the *"api-test.py"* file. In this file the *"test join"* function attempts to contact a node at *"localhost:10001"* to join the ring, which is not a valid contact point in this implementation. Therefore, this address has been changed to use the first available node from the list of running servers instead.

4 Conclusion

In this assignment a distributed hash table based on the Chord protocol was implemented. The system supports dynamic membership, allowing nodes to join and leave the network, giving the network the ability to recover from failures as well as storing and retrieving key-value pairs.

The implementation works as expected under various conditions such as nodes joining, nodes leaving, nodes crashing and system recovery. The provided API test, joining experiment test and the implemented tests in the *"controller.py"* file execute successfully, confirming that the Chord implementation works.